

Secure Exit

SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

KALAHSTHI RAMYA
192311149RA

CO3-AT3-Debugging and Optimization

[← Back](#)

QUESTIONS

Q1

ATM Withdrawal – Exception
Handling Debugging
10 marks

Q2

Hospital Registration –
NullPointerException
Debugging
10 marks

Q3

E-Commerce Product Search –
Array Exception Debugging
10 marks

Q4

Bank Account – Race Condition
Debugging
10 marks

Q5

Railway Reservation –
Synchronization Debugging
10 marks

5/5 answered

Q4

Bank Account – Race Condition
Debugging10
marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {  
  
    int balance = 10000;  
  
    void withdraw(int amount) {  
  
        if (balance >= amount) {  
  
            System.out.println(  
                Thread.currentThread().getName()  
                + " withdrawing " + amount);  
        }  
    }  
}
```

```
try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
    Thread.currentThread().getName()
    + " withdrawal successful");
}
else {
    System.out.println(
        Thread.currentThread().getName()
        + " insufficient balance");
}
}
}

class ATM extends Thread {

    BankAccount account;
    int amount;

    ATM(BankAccount account, int amount) {
        this.account = account;
        this.amount = amount;
    }

    public void run() {
        account.withdraw(amount);
    }
}

class BankApplication {

    public static void main(String[] args)
        throws InterruptedException {

        BankAccount account = new BankAccount();
```

```
ATM atm1 = new ATM(account, 7000);
ATM atm2 = new ATM(account, 6000);

atm1.setName("ATM-1");
atm2.setName("ATM-2");

atm1.start();
atm2.start();

atm1.join();
atm2.join();

System.out.println(
    "Final Balance = " + account.balance);
}
```

Correct the program so that only one withdrawal succeeds.

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2
3 public class Main{
4     public static void main(String[] args) {
5         BankAccount a = new BankAccount();
6
7         ATM a1 = new ATM(a, 7000);
8         ATM a2 = new ATM(a, 6000);
9
10        a1.setName("ATM 1");
11        a2.setName("ATM 2");
12
13        a1.start();
14        a2.start();
15
16        a1.join();
17        a2.join();
18
19        System.out.println("Final Balance =
```

```
20     }
21 }
22
23 class BankAccount{
24     int balance = 10000;
25
26     public synchronized void withdraw(int am
27         if(balance >= amount){
28         System.out.println(Thread.curren
29         try{
30             Thread.sleep(100);
31         }catch(InterruptedException e){
32             System.out.println("Interrup
33         }
34         balance = balance - amount;
35         System.out.println(Thread.curren
36     }else{
37         System.out.println(Thread.curren
38     }
39 }
40 }
41
42 class ATM extends Thread{
43     BankAccount account;
44     int amount;
45
46     ATM(BankAccount account,int amount){
47         this.account = account;
48         this.amount = amount;
49     }
50     public void run(){
51         account.withdraw(amount);
52     }
53 }
```



Test Input

stdin input (optional)

Output

```
ATM 1 withdrawing 7000  
ATM 1 withdrawal successful  
ATM 2 insufficient balance  
Final Balance = 3000
```

 Pending manual review

 Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.
